



UD3

INTRODUCCIÓN A JAVASCRIPT

3.2 INSTRUCCIONES DE CONTROL

MP_0373

Lenguaje de marcas y
Sistemas de gestión de
la información

Introducción

El flujo del programa es la línea que sigue el dispositivo ejecutando código. Si no ocurre nada que lo altere, el flujo empezará en la primera sentencia e irá ejecutando una a una cada sentencia de arriba a abajo hasta llegar al final.

Sin embargo, es complicado entender un programa sin que el flujo pueda variar. Por ejemplo, en una web de la previsión meteorológica habrá condiciones en el flujo: si hace buen tiempo ocurre una cosa y si hace mal tiempo ocurre otra cosa. De esta manera el flujo tiene caminos diferentes que recorre hasta llegar al final de la ejecución.

Estructuras de control condicional if/else

La estructura de control condicional if es la estructura más sencilla, con ella mediante una condición booleana el flujo puede tomar un camino específico para esa condición.

```
// Suponiendo la variable temperatura como temperatura actual
let mensaje = temperatura + "°";

// Si la temperatura es menor que 0, se añaden unos caracteres haciendo de
// copo de nieve
if (temperatura < 0) {
    mensaje = "***" + mensaje + "***";
}
console.log(mensaje);
```

La condición booleana aparece entre paréntesis (). Esta condición será evaluada por JavaScript y si el resultado es true, el flujo entrará a ejecutar las sentencias que hay dentro de las llaves {}, en caso contrario, saltará dichas sentencias.

Se puede complicar un poco más añadiendo otro camino para el flujo mediante la estructura if-else: Si se cumple la condición el flujo seguirá un camino y si no, se irá a otro camino:

```
// Suponiendo la variable edad declarada con un número que representa la edad
// de una persona
if (edad >= 18) {
```

```
    console.log("mayor de edad");  
  } else {  
    console.log("menor de edad");  
  }
```

Operador ternario

El operador ternario recrea la estructura de if-else pero de una manera muy simple y para casos en los que solo hay una sentencia para cada condición. Es simplemente una sintaxis diferente para esta estructura de control en el que se prescinde del if, else y de las llaves y se separa la condición de la primera sentencia con «?» y la primera sentencia de la segunda con «:».

```
// Todos los bloques siguientes imprimen por pantalla lo mismo  
// Utilizando estructura if-else  
if (edad >= 18) {  
  console.log("mayor de edad");  
} else {  
  console.log("menor de edad");  
}  
  
// Utilizando operador ternario  
(edad >=18) ? console.log("mayor de edad") : console.log("menor de edad");  
  
// Utilizando el operador ternario en una asignación  
const mensaje = (edad >=18) ? "mayor de edad" : "menor de edad";  
console.log(mensaje);  
// Utilizando el operador ternario directamente como parámetro de función  
console.log((edad >=18) ? "mayor de edad" : "menor de edad");
```

Estructuras de control condicional: switch

La estructura switch resuelve un flujo condicional cuando este tiene muchas opciones. Podría resolverse con un if-else muy grande, pero la estructura switch es más fácil de leer y mantener.

// Convierte una variable donde está el día de la semana en número en la palabra en castellano para ese día

```
let dia;
switch (diaNumero) {
  case 0:
    dia = "Lunes";
    break;
  case 1:
    dia = "Martes";
    break;
  case 2:
    dia = "Miércoles";
    break;
  case 3:
    dia = "Jueves";
    break;
  default:
    dia = "Error";
}
console.log(dia);
```

La sentencia break es parte de la estructura switch, es necesario añadirla ya que sin ella se evaluaría las sentencias del siguiente case, aunque la condición no fuese verdadera.

Por otro lado, default se puede utilizar como caso especial, el flujo evaluará las sentencias que estén dentro de él cuando ningún caso anterior haya sido seleccionado.

Estructuras de control iterativas: for / for in / for of

For

Estas estructuras es la forma más sencilla de manejar un bucle. Un bucle es una estructura de control que ejecuta una y otra vez un mismo conjunto de sentencias.

La estructura for funciona utilizando un índice (un número entero que va cambiando de valor) como control, de tal manera que se establece:

- ⊕ Un punto de inicio.
- ⊕ La manera en la que ese índice va a cambiar.
- ⊕ Un punto de final.

A cada vuelta se puede consultar el índice. Utilizando la sintaxis del for se indica esas condiciones para el bucle, dentro de los paréntesis () y separado por «;» se indica:

- ⊕ El índice que se va a utilizar.
- ⊕ La condición para la que el bucle seguirá repitiéndose.
- ⊕ Qué transformación se hace del índice a cada iteración (vuelta del bucle).

```
// En el siguiente bucle el índice empieza en el 0
// Seguirá repitiéndose mientras el índice sea menor que 10
// A cada vuelta, se sumará 1 al índice (i++ es lo mismo que i = i + 1)
```

```
//For para rellenar lista
for (i = 0; i < 10; i++) {
  console.log(i);
}
```

```
// Esto mostrará por consola:
// 0123456789
```

For in

Funciona de una manera similar al bucle for, sin embargo, en lugar de utilizar un índice, recorre un array o un objeto y deja disponible un índice correspondiente a cada uno de los elementos del array u objeto:

```
const arr = ["a", "b", "5"];
for (indice in arr) {
  console.log(indice); // Cada uno de los índices / posiciones del array
  console.log(arr[indice]); // Cada uno de los elementos que ocupan
posiciones en el array
}
// Se mostrará por consola
// 0 a 1 b 2 c
```

For of

Esta estructura itera un objeto iterable como un array y deja en cada vuelta del bucle disponible para una de las partes que componen ese objeto iterable:

```
let array = ["a", "b", "c"];
```

```
for (let value of array) {  
  value = value + "33";  
  console.log(value);  
}  
// a33 b33 c33
```

// Si el valor no se va a modificar dentro del bucle, se puede declarar la variable con const

```
let array = ["a", "b", "c"];  
  
for (const value of array) {  
  console.log(value);  
}  
// a b c
```

Estructuras de control iterativas: while / do-while

La estructura while como su nombre indica realiza un bucle mientras una condición se cumpla, en el momento que esa condición se evalúe como falsa, el bucle acabará.

```
let i = 0;  
while (i < 4) {  
  console.log(i);  
  i++;  
}
```

```
// Se mostrará por consola  
// 0 1 2 3 4
```

```
let i = 10;
```

```
while (i < 4) {  
  console.log(i);  
  i++;  
}
```

// No se mostrará nada por consola porque en la primera vuelta del bucle se evalúa (10 < 4) a por lo que no entra en el bucle

Similar al while, la estructura do-while que es similar con una salvedad: la primera vuelta se ejecutará siempre:

```
let i = 1;  
const n = 5;  
  
do {  
  console.log(i);  
  i++;  
} while(i <= n);
```

// Se mostrará por consola
// 1 2 3 4 5

Interrumpir o abandonar un bucle

La sentencia break sirve para terminar bucles, hacer que el flujo no respete el bucle y salte como si el bucle hubiese acabado.

La utilización de break debe hacerse en muy determinados casos, siempre es más recomendable diseñar correctamente un bucle antes de tener que recurrir a esta sentencia, a excepción de la sentencia switch donde la utilización de break es requerida.

Por ejemplo, el siguiente bucle está diseñado para ir del 1 al 5, sin embargo, en su ejecución cuando el valor del índice es 3, la sentencia break hace que el flujo salte, evitando el resto del bucle.

```
// Bucle con un índice del 1 al 5
```

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) {  
    break;  
  }  
  console.log(i);  
}  
  
// Se mostrará por consola  
// 1 2
```

Gestión de excepciones try-catch

Una excepción es un error que se produce (por la razón que sea) cuando un programa se está ejecutando, y salvo que se gestione esa excepción, la ejecución del programa no puede seguir.

Para manejar excepciones y que, aunque ocurra una el programa pueda seguir ejecutándose, se utiliza la estructura try/catch.

```
try {  
  console.log("pre error");  
  console.log(a);  
  console.log("post error");  
}  
  
catch(error) {  
  console.log('Error: ' + error);  
}  
  
// Mostará por consola  
// pre error  
// Error: ReferenceError: a is not defined
```

En el anterior ejemplo el flujo de ejecución entra en el try, cualquier excepción que ocurra dentro de ese bloque try será manejado por el bloque catch, por donde continuará la ejecución (saltando el resto de las sentencias que pudiese haber en el bloque try).

Como complemento al try-catch existe finally, gracias a finally se pueden agrupar sentencias de manera que se **ejecutarán tanto ocurra una excepción como si no**.

```
// Se produce excepción
try {
  console.log("pre error");
  console.log(a);
  console.log("post error");
}
catch(error) {
  console.log('Error: ' + error);
}
finally {
  console.log("finally");
}

// pre error
// Error: ReferenceError: a is not defined
// finally
// No se produce excepción
try {
  console.log("pre error");
  console.log("post error");
}
catch(error) {
  console.log('Error: ' + error);
}
finally {
  console.log("finally");
}

// pre error
// post error
// finally
```